

Separable NM-ROM: the Poisson and Burgers results

Accuracy, cost per resolution, and where the ROM stands against the full-order solver

Tunable-NM-ROM project

4 September 2026 — numbers final for these checkpoints (single seed, f64)

Every table is generated from the run JSONs (via the source reports named on each slide); nothing is typed by hand. Architecture slides are adapted from the 29 August study deck (sampled-quadrature arms removed); the rest is the measured results.

What was measured, in one slide

The model

A frozen decoder $u = G h(z)$ maps a small latent $z \in \mathbb{R}^K$ to the grid. The ROM solves the PDE for z by driving a *projected residual* $\Phi^\top r(u)$ to zero, with M test modes Φ .

The residual arms compared

- **full / oracle**: residual summed on the whole grid every iteration — the reference, cost $O(n)$.
- **quadrature-free (Poisson) / tensor (Burgers)**: residual precomputed into a small table — exact, no grid.
- Poisson also shows a **sampled** row (residual estimated at 256 chosen points) for contrast.

The three questions each PDE answers

1. **Accuracy** — is the sample-free arm as accurate as the full grid?
2. **Cost per resolution** — does the ROM's time stay flat as the grid N grows?
3. **Against the FOM** — at equal accuracy, is the ROM faster than a tuned classical solver?

How to read the numbers

Errors are relative L_2 against the full-order solution on held-out cases. Times are milliseconds per query (one Poisson solve, or one 50-step Burgers trajectory).
“Matched” FOM = the cheapest classical tolerance setting that is at least as accurate as the ROM.

The separable decoder

$$u(x; z) = bc(x) \langle g(x), h(z) \rangle = bc(x) \sum_{j=1}^R g_j(x) h_j(z)$$

- $g(x) \in \mathbb{R}^R$ — the **bank**: R spatial *shapes* (Fourier features $\rightarrow$ small MLP). Depends on x only.
- $h(z) \in \mathbb{R}^R$ — the **head**: MLP + linear skip from the latent to R *mixing amounts*. Depends on z only.
- $bc(x)$ — a fixed mask that makes the boundary condition hold exactly.
- $R = 32$ (Burgers), $R = 64\text{--}96$ (Poisson).

Why “separable” matters

The two tracks never see each other's input.

Evaluate the bank on any point set once and store it as a table

$$G \in \mathbb{R}^{n \times R}, \quad G_{ij} = bc(x_i) g_j(x_i).$$

Then the field on that point set is just

$$u(z) = G h(z) \quad (\text{table} \times \text{coefficient vector}).$$

The field is a mix of R fixed shapes; only the R mixing amounts depend on z . Everything downstream exploits exactly this.

Stage 1 — training the decoder (auto-decoder)

$$L_1 = \underbrace{\frac{\text{mean}_s \|G h(z_s) - u_s\|^2}{\text{mean} \|u\|^2}}_{\text{relative reconstruction}} + \lambda_{\text{orth}} \underbrace{\left\| \frac{G^\top G}{n s^2} - I \right\|_F^2}_{\text{keep the bank well-conditioned}}$$

Jointly optimise the bank g , the head h , and **one latent code** z_s **per training snapshot** u_s (there is no encoder — the codes are free variables). $\lambda_{\text{orth}} = 10^{-4}$; Adam, full batch, warm-up then cosine decay ($10^{-3} \rightarrow 10^{-5}$), 40 000 steps.

In words

- First term: “every training snapshot must be reconstructible from *some* 8-number code.”
- Second term: “the R shapes should be roughly orthogonal” — a conditioning nudge so the mixing amounts stay well-scaled; it does not change what the decoder can represent.

After stage 1

Bank, head, and codes are **frozen**. Nothing downstream ever changes the decoder. The training codes are discarded; the solver finds its own z at run time.

How the decoder is used online — it defines a manifold, it does not predict

Weak form. Pick M test functions $\Phi \in \mathbb{R}^{n \times M}$ (sine modes = exact eigenvectors of the discrete Laplacian, $\Delta \Phi = \Phi \Lambda$). The residual handed to the solver is the PDE residual projected onto them:

$$r(z) = \Phi^\top \mathcal{R}(G h(z)) \in \mathbb{R}^M, \quad M = 32 \text{ or } 64.$$

Solve. Each time step: damped Levenberg–Marquardt over the K latent unknowns, driving $r(z) \rightarrow 0$; needs r and $\partial r / \partial z$.

The cost question

$\Phi^\top \mathcal{R}(u)$ is a **sum over all n grid points**. Done naively, each of thousands of solver iterations would

1. decode $u = G h(z)$ on the grid ($n \times R$ flops),
2. apply the PDE operator on the grid,
3. sum against M test modes.

That is the full-order cost, every iteration.

The way out: regroup, so the grid disappears

Because $u = G h(z)$ is a mix of R fixed shapes, every sum over the grid can be done *once* for the shapes and stored.

Linear terms become an $M \times R$ matrix (Poisson, and the Burgers time and diffusion terms). **The quadratic term** of Burgers becomes an $M \times R \times R$ table — the tensor (next slides).

Poisson: the whole residual is a small table

$$\boxed{r(z) = W \odot [\Lambda B h(z) - f_M]} \quad \frac{\partial r}{\partial z} = W \odot \Lambda B \frac{\partial h}{\partial z}, \quad B = \Phi^\top G, \quad f_M = \Phi^\top f$$

Poisson: $-\Delta u = f$. Every term is linear in u .

- **Laplacian moved onto the test modes.** Because $\Delta \Phi = \Phi \Lambda$, $\Phi^\top (\Delta u) = (\Delta \Phi)^\top u = \Lambda \Phi^\top u$. The operator becomes M known numbers Λ — it never touches the field.
- **Source term projected once:** $f_M = \Phi^\top f$.
- $\partial h / \partial z$: autodiff of a $16 \rightarrow 64$ MLP ($16 \rightarrow 96$ at $N = 128$).
- W : fixed per-mode scaling that balances the rows.

What runs online

- the tiny head $h(z)$,
- a 64×64 matvec (64×96 at $N = 128$),
- the head's 64×16 Jacobian.

No grid. No decode. No sample points. No fitted weights. The field $G h(z)$ is formed once, at the very end, to write out the answer.

What “0 sample points” means

The sampled (NNLS) path estimates the same M numbers from 256 chosen points — and gets them 5–8 % wrong at solutions. The quadrature-free path computes them exactly and does not need the points at all.

Burgers: how the tensor is formed, and why it is needed

Backward-Euler step from z^n to z , $u = G h(z)$, upwind derivative $D^- u$ (valid while $u \geq 0$):

$$r(z) = W \odot \left[\underbrace{B(h(z) - h(z^n))}_{\text{time derivative, exact}} + \Delta t \left(\underbrace{\Phi^\top (u \odot D^- u)}_{\text{advection — the problem}} + \underbrace{\nu \wedge B h(z)}_{\text{diffusion, exact}} \right) \right], \quad B = \Phi^\top G$$

Why the linear trick stops

u and $D^- u$ both depend on z , so their product is not “a fixed matrix times $h(z)$ ”. Done directly, each solver iteration decodes u on the grid, applies the stencil, and sums against M modes: $O(n)$, the full-order cost.

Why it still collapses

$u = \sum_j h_j g_j$ and $D^- u = \sum_k h_k D^- g_k$, so

$$\Phi^\top (u \odot D^- u)_i = \sum_{j,k} h_j h_k \underbrace{\sum_x \Phi_{xi} G_{xj} (D^- G)_{xk}}_{T_{ijk}}$$

The grid sum sits inside T , which depends only on the frozen bank.

Building it, once

$T = \Phi^\top [G \otimes D^- G]$: an $M \times R \times R$ table (64^3 , 2 MB in 2D), one blocked contraction over the grid after stage 1. Symmetrised, $Q_{ijk} = T_{ijk} + T_{ikj}$, so that $q(h) = \frac{1}{2} h^\top Q h$, $\partial q / \partial h = Q h$.

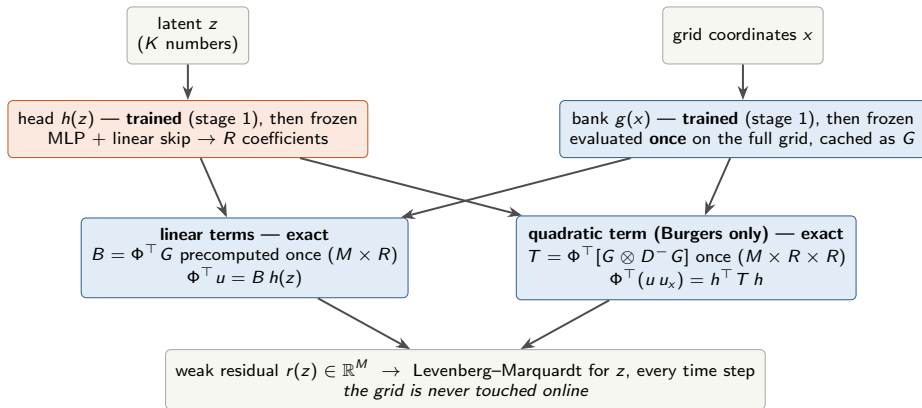
Using it, every iteration

the head $h(z)$, two $M \times R$ matvecs, and one MR^2 quadratic form. **No grid, no sample points, no fitted weights, nothing chosen offline.** Exact to round-off (f64 required).

The one condition

The upwind stencil switches direction on the sign of u ; T assumes it does not. Valid on non-negative states, which every family here has (truth asserted ≥ 0 at every N).

Architecture diagram — who is trained, who is frozen, what is precomputed



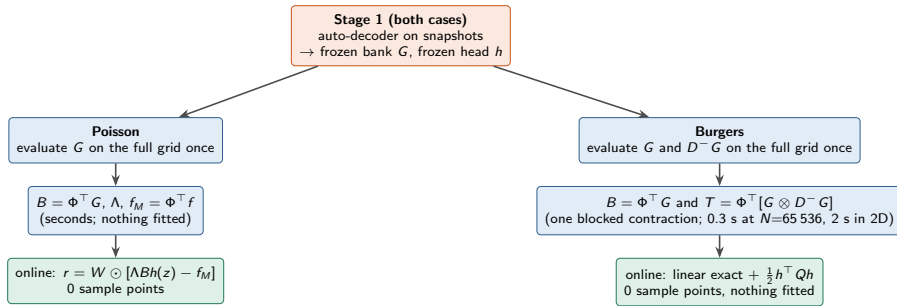
■ trained by gradient (stage 1), then frozen

■ precomputed once from the frozen bank, exact

Two trained pieces, both frozen after stage 1. The head and the bank are the only things learned. The solver never changes them; it only finds z .

Everything the solver touches is a small table. Poisson uses the left box only; Burgers uses both. In either case the grid appears once, offline, and never online.

After stage 1: what each case builds from the frozen decoder



■ gradient training ■ one-time exact precomputation from the frozen bank ■ direct evaluation online, no gradients, nothing fitted

The decoder training is identical in both columns; they differ only in what is *derived* from the frozen bank. A linear term is derived into a matrix, a quadratic term into a tensor. Nothing online ever sees the grid.

Poisson 2D — three residual paths through the same solver

N	residual path	sample pts	solve ms	solution err	resid. err vs full	grad. cos (min)	setup s
128	full grid	15 876	3.98	3.06e-2	0 (ref)	1.000	—
128	sampled (NNLS)	256	3.14	3.06e-2	7.88e-2	-0.608	28.6
128	quadrature-free	0	2.93	3.06e-2	4.23e-14	1.000	0.9
256	full grid	64 516	4.96	3.14e-2	0 (ref)	1.000	—
256	sampled (NNLS)	256	2.75	3.14e-2	6.21e-2	-0.602	37.3
256	quadrature-free	0	2.68	3.14e-2	3.61e-14	1.000	0.7
512	full grid	260 100	11.11	3.15e-2	0 (ref)	1.000	—
512	sampled (NNLS)	256	3.09	3.15e-2	5.50e-2	-0.653	25.4
512	quadrature-free	0	2.87	3.15e-2	3.25e-14	1.000	0.8

Online solve ms per held-out source at $\tau = 10^{-3}$, $K = 16$; all three paths through the identical solver on one A100-80GB. Solution error = relative L_2 against the FOM; residual error = relative deviation of the projected residual from the full-grid value at the solver's solutions; grad. cos = worst cosine between the path's gradient and the full-grid gradient; setup = one-time build of the path (NNLS fit or the $M \times R$ product).

Same answer. The solution error is identical down all three rows of each block: it is the decoder's floor, and the residual path does not touch it.

Exact, not sampled. The quadrature-free residual agrees with the full grid to $\sim 10^{-14}$; the sampled rule is 5–8 % off and its gradient can point the wrong way.

Flat and free. Full grid climbs 4 $\rightarrow$ 11 ms across N ; quadrature-free stays under 3 ms, and its setup is under a second instead of a 25–37 s NNLS fit.

Source: 2026-08-27-b1d-node-screening-and-poisson-qf.md.

Poisson 2D — the exactness gate

N	resid. rel. max (random z)	grad. rel. max (random z)	resid. rel. max (at solutions)	gate failures
128	1.33e-15	7.06e-16	9.99e-14	0
256	7.41e-16	1.12e-15	2.90e-13	0
512	7.34e-16	6.19e-16	1.37e-13	0

Gate Q: quadrature-free residual and gradient against the full grid, relative, in f64.

Residual and gradient of the quadrature-free path were checked against the full grid at random latent states and at every solver solution. All at machine precision, zero failures.

This is what “no sample points” means: the M projected residual values are *computed*, not estimated, so there is nothing to fit and nothing to get wrong.

Source: 2026-08-27-b1d-node-screening-and-poisson-qf.md.

Poisson 2D — against the classical solver

N	unknowns	ROM ms	ROM err	CG ms	CG err	who wins
64	3 844	2.06	3.75e-2	1.31	3.20e-2	CG 1.6×
128	15 876	3.09	3.06e-2	3.75	1.70e-2	ROM 1.2×
256	64 516	2.54	2.89e-2	8.16	1.10e-2	ROM 3.2×
512	260 100	3.06	3.15e-2	26.36	7.10e-3	ROM 8.6×
1024	1 044 484	3.50	3.48e-2	98.31	4.90e-3	ROM 28.1×

$K = 16$, held-out sources, $\tau = 10^{-3}$, same job and GPU. CG = cheapest unpreconditioned conjugate-gradient tolerance rung at least as accurate as the ROM (cheapest rung if none qualifies).

The ROM is flat; CG is not. ROM time stays at 2–3.5 ms from $N = 64$ to $N = 1024$ while conjugate gradient grows with the grid. The crossover is at $N = 128$.

Two caveats, stated plainly. The CG rung is *unpreconditioned*, so these speedups are soft. And Poisson with constant coefficients has an exact spectral direct solve (0.64 ms at $N = 1024$, error 7×10^{-15}) that beats both. The ROM's win is against *iterative* solvers.

Source: 2026-08-25-poisson-architecture-and-results.md.

Burgers 1D — the tensor arm lands on the full-grid oracle

N	full-grid (oracle) err	tensor err	tensor—oracle max per traj	e2e oracle ms	e2e tensor ms
128	5.01e-3	5.01e-3	1.2e-6	29.0	24.4
256	6.16e-3	6.16e-3	6.4e-7	35.0	29.2
512	4.88e-3	4.88e-3	2.4e-7	30.7	27.5
1024	5.54e-3	5.54e-3	2.3e-7	27.4	21.8

Rollout error = mean relative L_2 over 8 held-out trajectories $\times$ 51 states; e2e = end-to-end ms per 50-step trajectory (median of 8 traj $\times$ 5 reps), one A100, resolutions interleaved in one job.

Accuracy. Tensor = oracle per trajectory to $\leq 1.2 \times 10^{-6}$, with the grid never touched online. Flat across $8\times$ in resolution.

What the error is. About 5×10^{-3} at every N : the decoder's own floor. The residual adds nothing to it — the oracle and the tensor are the same solver seen through two implementations.

Source: 2026-08-29-b1d-tensor-sample-free-burgers.md.

Cost. 22–29 ms end to end, 10–20 % under the oracle. In 1D everything is launch-bound, so no arm's time moves with N .

Burgers 1D — cost per resolution, and against the FOM

arm	N range	solve ms	IC fit ms	e2e ms
oracle	128..4096	-0.0374	-0.0055	-0.0322
tensor	128..4096	-0.0772	-0.0011	-0.0493

Exponent p in $\text{ms} \sim N^p$, least-squares fit of $\log \text{ms}$ against $\log N$ inside one job on one GPU.

Flat in N — and so is everything else. Both arms' exponents are ≈ 0 over 128...4096: below $\sim 10^4$ points the GPU time is kernel launches, not arithmetic. Flatness is real; a 1D crossover is not.

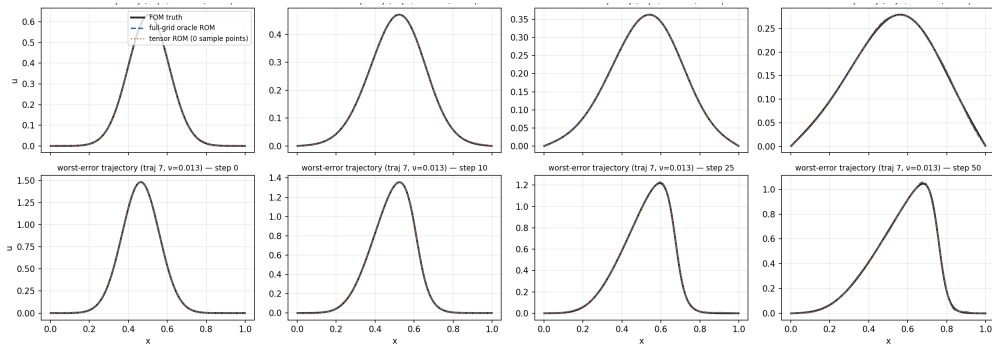
N	ROM e2e ms	FOM ms	ROM/FOM	ROM ms/traj (batch of 8)	ROM err
512	24.0	7.87	$3.0\times$	7.9	$5.19\text{e-}3$
4096	21.9	8.77	$2.5\times$	6.6	$4.89\text{e-}3$

An optimised 1D ROM rollout against the tridiagonal tolerance-Newton FOM at $\text{ntol } 10^{-3}$. Not iso-accuracy: the FOM is more accurate at every N . The FOM was not batched.

No 1D speedup. The tridiagonal FOM solves a trajectory in ~ 8 ms and is more accurate; the single-query ROM is $2.5\text{--}3\times$ slower. Only batched throughput (8 trajectories at once) brings the ROM to the FOM's per-trajectory time. (Timed on an earlier 1D rollout whose end-to-end time equals the tensor arm's to within a few ms.)

Sources: 2026-08-29-b1d-tensor-sample-free-burgers.md, 2026-08-28-b1d-rollout-optimization.md.

Burgers 1D — seeing it ($N = 512$)



FOM truth (black), full-grid oracle (blue dashed) and tensor ROM (orange dotted) at steps 0/10/25/50; top the median-error trajectory, bottom the worst. All curves coincide at this scale.

Burgers 2D — accuracy per arm ($K = 16, R = 64$)

N	full-grid err	tensor err	tensor—full max	tensor/full	same LM attempts
64	2.31e-2	2.31e-2	7.9e-5	1.00033	True
256	2.45e-2	2.45e-2	7.3e-6	1.00003	True
512	2.46e-2	2.46e-2	1.9e-6	1.00001	True
1024	6.38e-2	6.38e-2	2.0e-5	1.00006	True

$K = 16, R = 64, M = 64$. Rollout error = mean relative L_2 over 8 held-out trajectories $\times$ 51 states. *full* = advection summed on the whole grid every iteration; *tensor* = advection through the precomputed quadratic form.

Tensor = full at every N : same error to 5 digits, same stop reasons, same number of solver attempts.

The error is the decoder's floor. About 2.4×10^{-2} for the $K = 16$ decoders; a larger latent lowers it (not shown here).

Source: 2026-08-30-b2d-tensor-ladder.md.

The $N = 1024$ row is a weaker inherited decoder; its 6.4×10^{-2} is the checkpoint's floor, not the residual's.

Burgers 2D — cost per resolution

<i>N</i>	GPU	full-grid solve	tensor solve	tensor/full	IC fit	decode	tensor e2e
64	A100-40	32.6	27.5	0.845	2.0	0.40	29.6
256	A100-80	74.1	27.7	0.374	2.2	0.54	29.8
512	H200	97.3	20.0	0.205	1.6	0.49	21.1
1024	A100-80	670.0	27.1	0.040	4.8	1.15	32.6

Latent-solve ms per 50-step trajectory. **Each N ran on a different GPU:** the ratios are measured inside one job and are sound; the raw ms across rows mix cards and are not an exponent.

The $O(n)$ arm moves; the tensor does not.

Full-grid solve climbs $33 \rightarrow 670$ ms over $16\times$ in resolution. The tensor stays at 20–28 ms; its one-time build is about 2 s.

Mixed hardware. The four rungs ran on three different cards. Read the within-row tensor/full ratio; do not fit an exponent through the ms column.

Source: 2026-08-30-b2d-tensor-ladder.md.

Burgers 2D — against the FOM at matched accuracy ($K = 16$)

N	GPU	tensor err	tensor e2e	matched FOM err	matched FOM ms	paired ROM ms	paired FOM ms	speedup	tightest FOM err	tightest FOM ms
64	A100-40	2.31e-2	29.6	3.75e-4	12.4	29.6	12.4	0.42×	1.35e-4	16.3
256	A100-80	2.45e-2	29.8	3.54e-4	17.3	29.5	17.2	0.58×	1.33e-4	23.6
512	H200	2.46e-2	21.1	3.51e-4	21.1	21.5	21.0	0.98×	1.35e-4	27.4
1024	A100-80	6.38e-2	32.6	2.97e-2	120.0	34.1	117.7	3.45×	1.39e-4	230.7

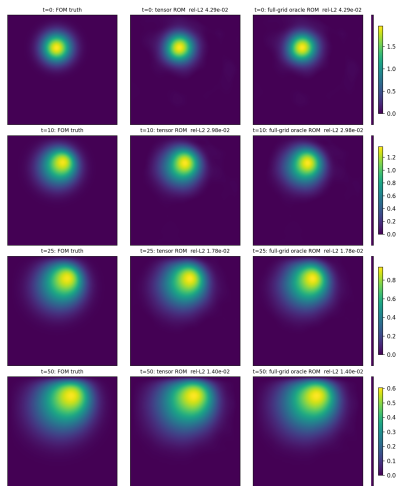
FOM = tolerance-Newton with the exact Helmholtz preconditioner, same GPU as the ROM. *matched* = cheapest rung at least as accurate as the tensor arm; *paired* = ROM and matched FOM timed alternately (AB/BA) in one job; *tightest* = most accurate rung run.

A real crossover, at the finest grid only. Below $N = 1024$ the preconditioned Newton FOM reaches the ROM's accuracy in 12–21 ms and wins. At $N = 1024$ its matched rung costs 118 ms and the tensor ROM is 3.45× faster.

But the FOM is far more accurate when asked. Its tightest rung reaches $\sim 1.4 \times 10^{-4}$ at every N , two orders below the ROM's floor. The ROM wins on time only at the accuracy its decoder allows.

Source: 2026-08-30-b2d-tensor-ladder.md.

Burgers 2D — seeing it ($N = 256$)



Truth, the full-grid ROM and the tensor ROM on one held-out trajectory; the two ROMs are indistinguishable at this scale.

The results in one table

	Accuracy of the sample-free arm	ROM cost vs N	Against the FOM at matched accuracy
Poisson 2D	= full grid to 10^{-14} ; solution error unchanged	flat, ~ 3 ms	wins vs unpreconditioned CG from $N = 128$; loses to the spectral direct solve
Burgers 1D	= full-grid oracle to 10^{-6}	flat, 22–29 ms (launch-bound)	loses $2.5\text{--}3\times$ to the tridiagonal FOM at every N
Burgers 2D	= full-grid oracle to 10^{-5}	flat, 20–28 ms (full grid grows to 670 ms)	loses below $N = 1024$; wins $3.45\times$ at $N = 1024$

What is established. The precomputed residual (quadrature-free for linear terms, tensor for the quadratic one) reproduces the full-grid ROM exactly and makes the online cost independent of the grid.

What is not. A speed win over a well-tuned classical solver exists only where the grid is large enough for the FOM to leave the launch floor: Burgers 2D at 1024^2 . Everything here is single-seed.

Glossary

FOM full-order model: the classical solver on the whole grid.

ROM reduced-order model: solves for the latent z only.

N grid points per side; unknowns $\approx N^2$ in 2D.

K, R, M latent size, decoder bank width, number of test modes.

full / oracle residual summed on the whole grid every iteration.

sampled (Poisson row only) residual estimated at 256 points chosen by non-negative least squares (NNLS).

$\Phi, B, T/Q$ test modes; the linear-term matrix $\Phi^\top G$; the advection tensor and its symmetrised form.

upwind D^- backward difference, the stable derivative for $u \geq 0$.

quadrature-free / tensor residual precomputed into a small table; exact, no grid, no points.

e2e end to end: initial-condition fit, latent solve, and final decode.

matched rung cheapest FOM tolerance at least as accurate as the ROM.

paired / AB/BA ROM and FOM timed alternately in one job on one GPU.

held-out cases never seen in training.

CG conjugate gradient, here unpreconditioned.

launch-bound time dominated by GPU kernel launches, not arithmetic.